# Sri Lanka Institute of Information Technology


## Lab Submission

## Lab sheet No 05

**Exercise 03**

**IT25100826**

**Udumullage S.H.U.**

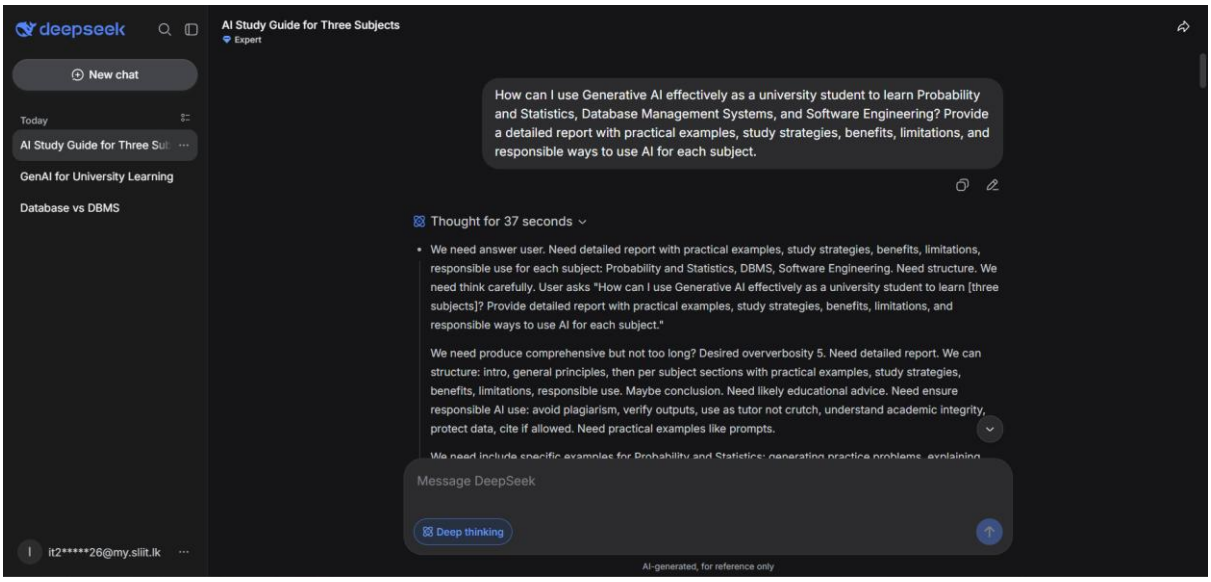
**IT2011 - Artificial Intelligence and Machine Learning**

B.Sc. (Hons) in Information Technology

**Use the following large language models, enabling the reasoning model of the platform and generate a detail report on "How you can use generative AI as a university student to learn subjects like probability and statistics, data base management systems and software engineering"**

1. https://chat.deepseek.com - Deepthing R1

https://chat.deepseek.com/share/0nv8de33s6jguendyd




We need answer user. Need detailed report with practical examples, study strategies, benefits, limitations, responsible use for each subject: Probability and Statistics, DBMS, Software Engineering. Need structure. We need think carefully. User asks "How can I use Generative AI effectively as a university student to learn [three subjects]? Provide detailed report with practical examples, study strategies, benefits, limitations, and responsible ways to use AI for each subject."

We need produce comprehensive but not too long? Desired oververbosity 5. Need detailed report. We can structure: intro, general principles, then per subject sections with practical examples, study strategies, benefits, limitations, responsible use. Maybe conclusion. Need likely educational advice. Need ensure responsible AI use: avoid plagiarism, verify outputs, use as tutor not crutch, understand academic integrity, protect data, cite if allowed. Need practical examples like prompts.

We need include specific examples for Probability and Statistics: generating practice problems, explaining concepts like Bayes theorem, simulating random variables maybe Python code, checking derivations, interpreting distributions. DBMS: SQL query generation, ER diagram design, normalization explanation, transaction/concurrency examples, query optimization, practicing SQL, comparing NoSQL. Software Engineering: requirements

analysis, UML diagrams, design patterns, code review, testing strategies, project management, debugging, generating sample code with explanations, refactoring.

Need include limitations: hallucination, incorrect math/code, overreliance, outdated info, privacy, academic misconduct. Responsible use: use as Socratic tutor, ask for explanations, verify with textbooks, don't submit AI output as own, disclose if required, use university policies.

Need maybe include study strategies like active recall, spaced repetition, interleaving, Feynman technique, project-based learning. Could integrate.

Let's craft answer. Need maybe mention not all universities allow AI; check course policies. Use AI to supplement not replace. Need maybe include prompt templates.

We need produce final answer in report format with headings. We can include a table maybe. Ensure clear.

Let's draft.

Potential outline:

Title: Using Generative AI Effectively for Probability & Statistics, Database Management Systems, and Software Engineering

1. Introduction
    - Gen AI as adaptive tutor, practice generator, code assistant
    - Core principles: verify, use as supplement, maintain academic integrity, protect privacy, avoid overreliance.
2. Probability and Statistics
    - Practical examples:
        - Explain concepts: "Explain Central Limit Theorem with a simple example, then test me with a question."
        - Step-by-step problem solving: "Solve this Bayes theorem problem step by step, explaining each step."
        - Generate practice problems: "Create 5 problems on Poisson distribution with solutions hidden."
        - Simulate with code: "Write Python code to simulate law of large numbers."
        - Interpret results: "What does p-value mean in plain English? Give an example."

- Study strategies:
  - Use AI for formative assessment, not just answers.
  - Create concept maps and compare.
  - Use spaced repetition with generated flashcards.
  - Work through derivations and ask AI to check.
- Benefits: personalized explanations, unlimited practice, immediate feedback, visualization code.
- Limitations: may make arithmetic errors or misapply formulas; can provide plausible wrong explanations; may not align with course notation; cannot replace statistical intuition.
- Responsible use: ask for reasoning not just final; cross-check with textbook/software; don't use AI during closed-book exams; if using for assignments, check policy; cite if allowed.

3. Database Management Systems
   - Practical examples:
     - SQL practice: "Generate a table schema for a library system and write queries for X."
     - Explain normalization: "Explain 1NF, 2NF, 3NF with examples and identify anomalies."
     - ER diagram design: "Given business rules, create an ER diagram description or Mermaid code."
     - Query optimization: "Explain how indexing works and why this query is slow."
     - Transactions: "Explain ACID properties with a banking transaction example."
     - Compare models: "Compare relational and document databases for a blog application."
   - Study strategies:
     - Use AI to generate datasets and SQL exercises of increasing difficulty.
     - Design schemas and ask AI to critique normalization.
     - Practice translating requirements to SQL and ERD.

        - Debug SQL errors by pasting query and error.
        - Ask AI to act as a DBMS interviewer.
    - Benefits: interactive SQL tutor, instant feedback on syntax, help with complex joins/subqueries, generate test data.
    - Limitations: SQL syntax variations between DBMS; may suggest inefficient queries; can hallucinate functions; cannot replace hands-on DBMS experience; may not handle large data context.
    - Responsible use: run queries on actual DBMS to verify; understand execution plans yourself; don't copy schema designs without understanding; use for learning not for graded work unless permitted.
4. Software Engineering
    - Practical examples:
        - Requirements: "Help me write user stories for a ride-sharing app."
        - Design: "Explain MVC pattern with example; draw a class diagram in Mermaid for a library system."
        - Code review: "Review this code for readability and possible bugs."
        - Testing: "Generate unit test cases for this function; explain boundary value analysis."
        - Debugging: "Here is an error trace, what could be causing it?"
        - Refactoring: "Suggest refactoring for this function to improve cohesion."
        - Project management: "Create a sprint backlog from these requirements."
    - Study strategies:
        - Use AI to generate mini-project ideas and requirements.
        - Practice design patterns by asking for scenarios and then implementing.
        - Conduct AI-assisted code reviews on your own code.
        - Use AI to generate test cases and compare with your own.
        - Ask AI to explain architecture trade-offs.
    - Benefits: broad knowledge of tools/practices, code examples, design feedback, saves time on boilerplate, helps learn new languages/frameworks.

- Limitations: code may be insecure/outdated/buggy; may oversimplify architecture; cannot understand team dynamics; may produce non-idiomatic code; ethical concerns if used to complete assignments; may hallucinate APIs.
- Responsible use: treat generated code as draft; review line by line; follow course academic integrity; don't use AI for graded coding assignments if prohibited; use for learning concepts and generating practice; cite if allowed; avoid sharing proprietary code.

5. General Responsible Use Framework
    - Verify, don't trust.
    - Use as tutor, not oracle.
    - Maintain academic integrity: know policies, cite, disclose.
    - Protect privacy: don't paste personal or confidential data.
    - Develop metacognition: ask yourself if you can solve without AI.
    - Combine with textbooks, lectures, office hours, peers.
6. Conclusion
    - Gen AI can enhance learning if used deliberately and critically.

Need maybe include a table of sample prompts per subject? Could include in each section. Need not too huge but detailed.

Let's ensure answer satisfies "detailed report with practical examples, study strategies, benefits, limitations, and responsible ways to use AI for each subject." We'll provide exactly.

Could include "Example prompts" in boxes. Since plain text, use quotes.

Need maybe mention "Generative AI" could be ChatGPT, Claude, Gemini, Copilot etc. Keep generic.

Let's write final answer with headings. Need maybe moderate length but detailed. We can include bullet points.

Let's compose elegantly.

## Using Generative AI Effectively for Probability & Statistics, Database Management Systems, and Software Engineering

Generative AI tools—such as ChatGPT, Claude, Gemini, and GitHub Copilot—can act as on-demand tutors, practice-problem generators, code reviewers, and study partners. However, they are not replacements for textbooks, lectures, hands-on practice, or your own

critical thinking. This report outlines practical strategies, examples, benefits, limitations, and responsible-use guidelines for three university subjects.

---

## 1. Probability and Statistics

### Practical Examples

- **Concept explanations**
  *Prompt:* "Explain the Central Limit Theorem in simple terms and give a real-world example. Then ask me one conceptual question to check my understanding."
- **Step-by-step problem solving**
  *Prompt:* "Solve this Bayes' theorem problem step by step, explaining each step: A test for a disease is 95% accurate. The disease affects 1% of the population. If a person tests positive, what is the probability they have the disease?"
- **Generating practice problems**
  *Prompt:* "Generate five practice problems on Poisson distributions at increasing difficulty. Do not show the solutions immediately; let me attempt them first."
- **Simulating random phenomena with code**
  *Prompt:* "Write Python code to simulate the law of large numbers using a biased coin. Include a plot of the running average."
- **Interpreting statistical output**
  *Prompt:* "Explain what a p-value of 0.03 means in plain language. What are common misinterpretations?"

### Study Strategies

- Use AI as a **formative assessment tool**: ask it to generate quizzes, then check your answers.
- Ask AI to **explain the same concept in multiple ways**—visually, with formulas, with analogies—until it clicks.
- Create **flashcards** from AI-generated explanations and use spaced-repetition software.
- Work through textbook problems yourself first, then use AI to **verify your reasoning** or identify where you went wrong.
- Ask AI to **derive formulas step by step** and then try to reproduce the derivation without looking.

### Benefits

- Unlimited, personalized practice problems.
- Immediate feedback on conceptual misunderstandings.
- Ability to generate visualizations and simulations to build intuition.
- Plain-language explanations that complement mathematical notation.

### Limitations

- AI can make **arithmetic or algebraic mistakes**, especially in multi-step derivations.
- It may **misapply statistical methods** or use different notation than your course.
- It can produce **plausible but incorrect explanations**—especially on subtle topics like p-values, confidence intervals, and causality.
- It cannot replace the intuition gained from actually working with real data.

### Responsible Use

- Always **verify final answers** with a calculator, statistical software (R, Python), or your textbook.
- Ask for **reasoning and intermediate steps**, not just final answers.
- Do not use AI during closed-book exams or quizzes unless explicitly permitted.
- If you use AI to help with a graded assignment, **check your instructor's policy** and cite if required.
- Avoid pasting sensitive or private data into public AI tools.

---

## 2. Database Management Systems (DBMS)

### Practical Examples

- **SQL practice**
  *Prompt:* "Create a schema for a library database with tables for books, members, and loans. Write SQL queries to find overdue books and the most borrowed authors."
- **Normalization exercises**
  *Prompt:* "Explain 1NF, 2NF, and 3NF with examples. Then give me an unnormalized table and ask me to normalize it to 3NF."
- **ER diagram design**
  *Prompt:* "Given these business rules for a university enrollment system, describe an

Entity-Relationship diagram. Include entities, attributes, relationships, and cardinalities. You can use Mermaid syntax if helpful."

- **Query optimization**
  *Prompt:* "Why might this SQL query be slow? Explain how indexes work and suggest improvements."
  *(Paste the query.)*
- **Transaction and concurrency concepts**
  *Prompt:* "Explain ACID properties with a banking transaction example. What is a deadlock, and how can it be avoided?"
- **Comparing database models**
  *Prompt:* "Compare relational databases and document databases for a social media application. What are the trade-offs?"

## Study Strategies

- Use AI to **generate sample databases** and SQL exercises of increasing complexity.
- Design a schema, then ask AI to **critique it** for normalization, redundancy, or missing constraints.
- Practice **translating plain-English requirements** into SQL and ER diagrams.
- Debug your own SQL by pasting the query and error message: "Here is my SQL query and error. What is wrong and how do I fix it?"
- Use AI as a **mock interviewer**: "Ask me five DBMS interview questions, then evaluate my answers."

## Benefits

- Interactive SQL tutor with instant syntax feedback.
- Help with complex joins, subqueries, and window functions.
- Ability to generate realistic test data and schemas.
- Clear explanations of abstract concepts like normalization, indexing, and transactions.

## Limitations

- SQL **syntax varies** between MySQL, PostgreSQL, SQL Server, SQLite, and Oracle; AI may not match your DBMS.
- It may suggest **inefficient queries** or omit important edge cases.
- It can **hallucinate functions** that don't exist in your database system.

- It cannot replace hands-on experience with an actual DBMS—execution plans, performance tuning, and concurrency behave differently in real systems.

### Responsible Use

- Always **run AI-generated SQL** in a safe, local database to verify it works.
- Try to **explain the query yourself** before asking AI—struggling first deepens learning.
- Do not submit AI-generated schema designs or SQL as your own for graded work unless allowed.
- If your course allows AI assistance, **cite the tool** and document how you used it.
- Avoid pasting real or confidential database schemas/data into public tools.

---

## 3. Software Engineering

### Practical Examples

- **Requirements analysis**
  *Prompt:* "Help me write user stories and acceptance criteria for a ride-sharing app. Focus on the rider and driver roles."
- **Software design and architecture**
  *Prompt:* "Explain the Model-View-Controller pattern with an example. Then create a class diagram in Mermaid for a simple online bookstore."
- **Code review**
  *Prompt:* "Review this Python function for readability, possible bugs, and edge cases. Suggest improvements without rewriting it completely."
  *(Paste code.)*
- **Testing**
  *Prompt:* "Generate unit test cases for this function using pytest. Include normal, boundary, and error conditions."
  *(Paste function.)*
- **Debugging**
  *Prompt:* "Here is a stack trace from my Java program. What could be causing this NullPointerException, and how should I investigate?"
  *(Paste stack trace.)*
- **Refactoring**
  *Prompt:* "Suggest how to refactor this long method to improve cohesion and reduce duplication. Explain the trade-offs of your approach."

- **Project management**
  *Prompt:* "Create a sprint backlog with user stories, tasks, and priorities for a two-week sprint to build a login and registration feature."

### Study Strategies

- Use AI to **generate mini-project ideas** and then implement them yourself.
- Practice **design patterns** by asking AI for a scenario, then implementing the pattern without looking at AI code.
- Conduct **AI-assisted code reviews** on your own code after you've written it.
- Generate **test cases** with AI, but also write your own—compare coverage.
- Ask AI to **explain architecture trade-offs**: "Compare microservices and monoliths for a small startup."
- Use AI as a **rubber duck** to talk through design decisions.

### Benefits

- Broad exposure to tools, frameworks, and best practices.
- Quick generation of boilerplate code, templates, and configuration files.
- Immediate feedback on code style, potential bugs, and test cases.
- Help with understanding unfamiliar languages or codebases.

### Limitations

- Generated code may be **insecure, outdated, or non-idiomatic**.
- It may **hallucinate APIs** or use deprecated libraries.
- AI cannot understand **team dynamics, legacy constraints, or real-world performance**.
- It may oversimplify complex architectural decisions.
- Overreliance can prevent you from developing debugging and design skills.

### Responsible Use

- Treat AI-generated code as a **draft, not a final solution**—review every line.
- Never submit AI-generated code for graded assignments if it violates your course's **academic integrity policy**.
- If allowed, **cite AI assistance** and describe how you used it.
- Avoid sharing **proprietary or confidential code** with public AI tools.

- Use AI to **learn concepts and generate practice**, but write your own code for assessments.

---

### General Responsible Use Framework

| Principle | Action |
|---|---|
| **Verify** | Always check AI output against textbooks, official documentation, o running code. |
| **Use as tutor, not oracle** | Ask for explanations, hints, and feedback; avoid asking for complete solutions without understanding. |
| **Maintain academic integrity** | Know your course policies. Cite and disclose AI use where required. |
| **Protect privacy** | Do not paste personal, confidential, or proprietary data into public AI tools. |
| **Develop metacognition** | Regularly ask yourself: “Could I solve this without AI? Do I understand the reasoning?” |
| **Combine with traditional learning** | Use AI alongside lectures, office hours, study groups, and hands-on practice. |

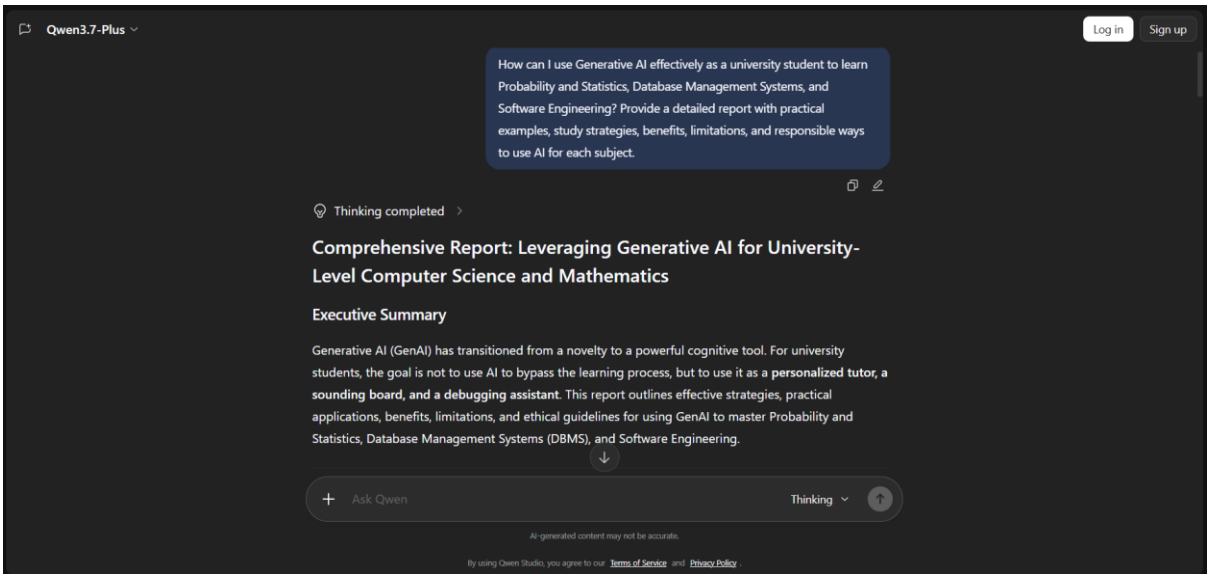
---

### Conclusion

Generative AI can significantly enhance your learning in Probability and Statistics, Database Management Systems, and Software Engineering—if used deliberately and critically. Use it to generate practice, explain difficult concepts, debug, and review your work. But always verify its output, follow academic integrity rules, and ensure you are building your own understanding. The goal is not to let AI do the work for you, but to use it as a tool that accelerates and deepens your learning.

2. **https://chat.qwen.ai - Thinking**

https://chat.qwen.ai/c/guest




Thinking completed

## Comprehensive Report: Leveraging Generative AI for University-Level Computer Science and Mathematics

### Executive Summary

Generative AI (GenAI) has transitioned from a novelty to a powerful cognitive tool. For university students, the goal is not to use AI to bypass the learning process, but to use it as a **personalized tutor, a sounding board, and a debugging assistant**. This report outlines effective strategies, practical applications, benefits, limitations, and ethical guidelines for using GenAI to master Probability and Statistics, Database Management Systems (DBMS), and Software Engineering.

---

### 1. Probability and Statistics (P&S)

Probability and Statistics requires a blend of mathematical rigor, logical reasoning, and real-world intuition. GenAI excels at building intuition and explaining abstract concepts.

#### Study Strategies

- **The Socratic Method:** Instead of asking for the answer, ask the AI to guide you through the problem step-by-step.

- **Analogy Generation:** Use AI to translate dense mathematical theorems into intuitive, real-world analogies.
- **Code-Assisted Statistics:** Use AI to write Python (using pandas, numpy, scipy) or R code to simulate distributions and visualize data, bridging the gap between theory and practice.

### Practical Examples (Prompts)

- **Conceptual Understanding:** *"Explain the Central Limit Theorem to me using a real-world analogy involving a coffee shop. Keep the math to a minimum and focus on the intuition."*
- **Guided Problem Solving:** *"I am trying to solve this Bayes' Theorem problem: [Insert Problem]. Do NOT give me the final answer. Instead, ask me a guiding question to help me identify the prior probability and the likelihood."*
- **Simulation & Visualization:** *"Write a Python script using matplotlib and scipy.stats to simulate 10,000 rolls of a biased die. Plot the distribution and calculate the empirical mean and variance."*

### Benefits

- **Demystifies Abstraction:** Turns dry textbook definitions into relatable scenarios.
- **Infinite Practice:** Can generate endless variations of a specific type of problem (e.g., Poisson distributions) with varying difficulty levels.
- **Bridges Math and Code:** Helps translate statistical formulas into executable code.

### Limitations

- **Calculation Hallucinations:** LLMs are notoriously bad at raw arithmetic. They might set up the equation perfectly but fail at the final calculation.
- **Overcomplication:** AI might use advanced mathematical jargon when a simpler explanation is needed.

### Responsible Use

- **Verify the Math:** Always do the final arithmetic yourself or use a computational tool (like WolframAlpha or Python) to verify the AI's numerical answers.
- **Focus on the "Why":** Use AI to understand the setup of the problem (e.g., "Is this a combination or permutation?"), not just to get the final number.

---

## 2. Database Management Systems (DBMS)

DBMS involves logical design, query optimization, normalization, and understanding transaction management. GenAI acts as an excellent pair-programmer and design reviewer.

### Study Strategies

- **Iterative Schema Design:** Treat the AI as a senior Database Administrator (DBA) to critique your Entity-Relationship (ER) designs.
- **Query Debugging:** Use AI to explain why your SQL queries are failing or performing poorly.
- **Visualizing Data:** Use AI to generate text-based diagrams (like Mermaid.js) for ER diagrams and relational schemas.

### Practical Examples (Prompts)

- **Requirements Gathering & Design:** *"Act as a Senior DBA. I need to design a relational database for a university course registration system. Ask me 3-5 clarifying questions about the business rules before we start designing the schema."*
- **SQL Debugging & Optimization:** *"Here is my SQL query to find the top 3 students by GPA in each department. It is returning the wrong results. [Insert Query and Table Schema]. Explain my logical error and provide a corrected query using window functions."*
- **Normalization Practice:** *"Give me an unnormalized relation with at least 5 attributes and some multi-valued dependencies. Walk me through normalizing it to 3NF, asking me to identify the functional dependencies at each step."*

### Benefits

- **Instant SQL Feedback:** Drastically reduces the time spent debugging syntax errors or logical flaws in complex joins.
- **Visual Aids:** Can generate Mermaid.js code to instantly render ER diagrams and flowcharts.
- **Contextual Learning:** Explains database theories (like ACID properties or isolation levels) in the context of specific database engines (PostgreSQL, MySQL).

### Limitations

- **Dialect Confusion:** AI might mix up syntax between different SQL dialects (e.g., using T-SQL syntax in a MySQL environment).
- **Inefficient Queries:** AI might write queries that work but are highly inefficient (e.g., using correlated subqueries instead of JOINs).

### Responsible Use

- **Test Locally:** Never assume AI-generated SQL is perfect. Always test it against a local database instance.
- **Understand the Execution Plan:** Don't just accept the AI's query; ask it to explain the *execution plan* and why it chose a specific join strategy to ensure you are learning optimization.

---

## 3. Software Engineering (SE)

Software Engineering is less about writing raw code and more about system design, methodologies (Agile/Scrum), architecture, testing, and teamwork. GenAI is a powerful tool for simulating scenarios and generating boilerplate.

### Study Strategies

- **Role-Playing Methodologies:** Simulate Agile ceremonies (Sprint Planning, Retrospectives) with the AI to practice soft skills and process understanding.
- **Architecture Brainstorming:** Use AI to compare architectural patterns (Microservices vs. Monolith, MVC vs. MVVM) for specific use cases.
- **Test-Driven Development (TDD):** Have the AI write the test cases first, then you write the implementation code.

### Practical Examples (Prompts)

- **Agile Simulation:** *"Let's roleplay a Sprint Retrospective. You are a backend developer who is frustrated because the frontend team keeps changing the API contract without notice. I am the Scrum Master. Start the conversation."*
- **Architecture Design:** *"I am designing an e-commerce platform that expects high traffic during holiday sales. Compare a Monolithic architecture vs. a Microservices architecture for this specific use case. Provide a text-based diagram using Mermaid.js for the Microservices approach."*
- **Design Patterns & Refactoring:** *"Here is a messy Java class that handles user authentication, password hashing, and email sending. Refactor this code to apply the Single Responsibility Principle and the Strategy Pattern. Explain the changes you made."*

### Benefits

- **Safe Environment for Soft Skills:** Allows students to practice difficult conversations (e.g., handling a underperforming team member in a simulation) without real-world consequences.
- **Rapid Prototyping:** Generates boilerplate code, Dockerfiles, and CI/CD pipeline configurations, allowing you to focus on high-level architecture.
- **Design Pattern Application:** Shows exactly *how* and *when* to apply abstract design patterns in real code.

### Limitations

- **Lack of Business Context:** AI doesn't understand the specific constraints of your university project (e.g., strict deadlines, specific grading rubrics, team skill levels).
- **Over-engineering:** AI tends to suggest enterprise-grade, highly complex solutions for what should be simple academic projects.

### Responsible Use

- **Maintain Academic Integrity:** If you are working on a group project, do not use AI to write the final documentation or code that you submit as your own work without proper citation.
- **Focus on the "Why":** When AI suggests an architectural pattern, ensure you can articulate *why* it is better than the alternative for your specific project constraints.

---

### General Guidelines for Responsible AI Use in University

To ensure you are actually learning and maintaining academic integrity, adhere to the following golden rules:

1. **The "Blank Page" Rule:** Always attempt the problem, write the first draft of the code, or draw the initial schema *yourself* before asking the AI for help. AI should be used to unblock you, not to start the work for you.
2. **Verify and Cite:** AI makes mistakes. Treat AI output as a first draft from a junior colleague—review it critically. If your university allows AI use, cite it properly (e.g., "Schema design assisted by ChatGPT-4 on [Date]").
3. **Protect Your Privacy:** Do not paste unpublished research data, proprietary company code (if doing an internship), or personally identifiable information (PII) into public LLMs.

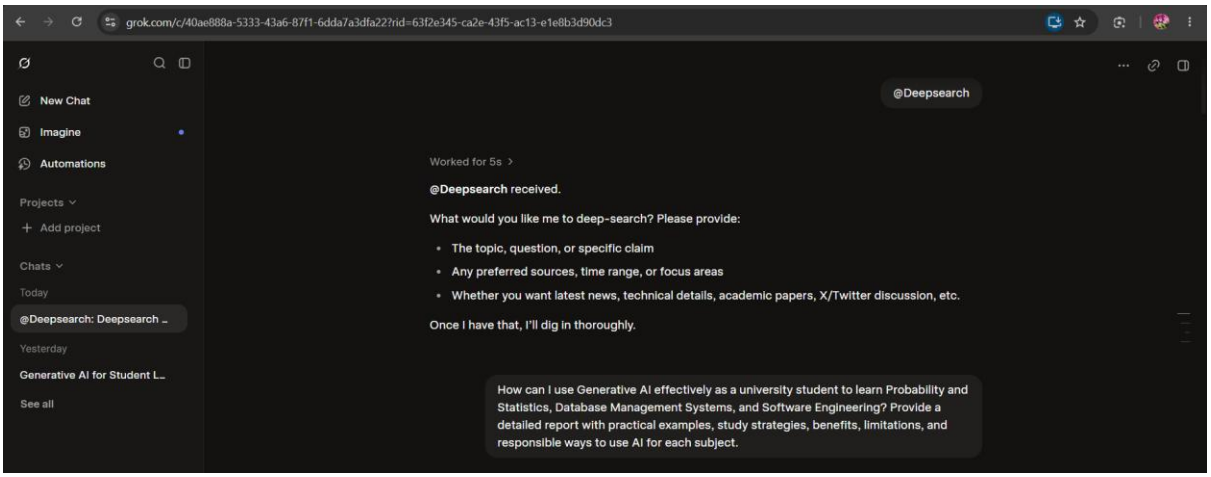
4. **Prompt Engineering is a Skill:** Learn to write better prompts. Include context, specify the persona you want the AI to adopt, and define the exact format of the output you need.
5. **Check Your Syllabus:** Always defer to your professor's specific policies regarding AI. Some may ban it entirely; others may encourage it for brainstorming but ban it for final submissions.

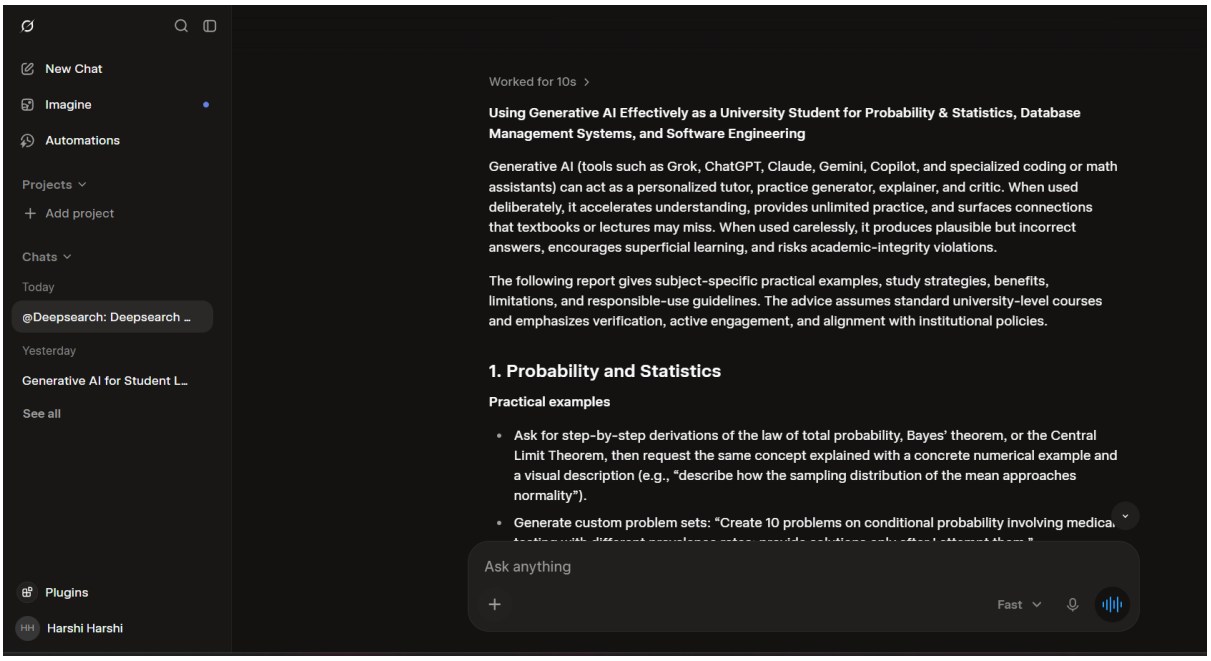
## Conclusion

Generative AI is not a replacement for the cognitive struggle required to learn complex subjects like P&S, DBMS, and SE. In fact, if used to bypass that struggle, it will result in a superficial understanding. However, when used as an interactive tutor, a debugging companion, and a simulator for real-world scenarios, GenAI can significantly accelerate your learning curve, deepen your conceptual understanding, and better prepare you for your future career in tech.

3. **https://x.com/i/grok - DeepSearch**

https://grok.com/share/c2hhcmQtMw_d32493b8-7feb-49d5-9506-8b1eae40ffbf






## Using Generative AI Effectively as a University Student for Probability & Statistics, Database Management Systems, and Software Engineering

Generative AI (tools such as Grok, ChatGPT, Claude, Gemini, Copilot, and specialized coding or math assistants) can act as a personalized tutor, practice generator, explainer, and critic. When used deliberately, it accelerates understanding, provides unlimited practice, and surfaces connections that textbooks or lectures may miss. When used carelessly, it produces plausible but incorrect answers, encourages superficial learning, and risks academic-integrity violations.

The following report gives subject-specific practical examples, study strategies, benefits, limitations, and responsible-use guidelines. The advice assumes standard university-level courses and emphasizes verification, active engagement, and alignment with institutional policies.

### 1. Probability and Statistics

### Practical examples

- Ask for step-by-step derivations of the law of total probability, Bayes' theorem, or the Central Limit Theorem, then request the same concept explained with a concrete numerical example and a visual description (e.g., "describe how the sampling distribution of the mean approaches normality").
- Generate custom problem sets: "Create 10 problems on conditional probability involving medical testing with different prevalence rates; provide solutions only after I attempt them."

- Simulate data and interpret results: “Generate a Python or R snippet that draws 10,000 samples from a binomial distribution and plots the sampling distribution of the sample proportion. Explain what the plot shows about the CLT.”
- Critique your own work: Paste a solution you wrote and ask the AI to identify conceptual errors or suggest alternative approaches (e.g., using generating functions versus direct computation).
- Explore real-world applications: “Walk me through a hypothesis test for a difference in means using a published dataset summary, including assumptions, p-value interpretation, and effect-size discussion.”

### Study strategies

- Concept mastery loop: Read the textbook/lecture notes → ask AI for a plain-language + mathematical explanation → solve 3–5 AI-generated problems without looking at solutions → compare your answers → ask for targeted clarification on mistakes.
- Spaced practice: Have the AI create a weekly mixed quiz covering recent and earlier topics; review explanations only after attempting.
- Visualization and intuition: Request multiple analogies or geometric interpretations (e.g., Venn diagrams for probability, density curves for continuous distributions).
- Code-assisted verification: Use AI-generated short scripts (Python/R/MATLAB) to check analytical results numerically; never treat the code as a black box—read and modify it.
- Exam simulation: Ask for timed problem sets that mirror past exam formats, then debrief with the AI on strategy and common pitfalls.

### Benefits

- Unlimited, adaptive practice problems tailored to your weak areas.
- Instant alternative explanations when a textbook proof feels opaque.
- Ability to explore “what-if” scenarios and sensitivity of results quickly.
- Bridging theory and computation (analytical derivation + simulation check).

### Limitations

- AI can produce subtle algebraic or conceptual errors, especially in multi-step proofs or edge cases (e.g., improper handling of continuous vs. discrete, or incorrect application of independence assumptions).

- Over-reliance on generated solutions reduces the struggle that builds deep statistical intuition.
- Visualizations are described rather than rendered (unless the tool supports code execution or image generation); you still need to run plots yourself.
- AI may oversimplify assumptions or present frequentist and Bayesian viewpoints without clear distinction.

### Responsible use

- Always attempt problems yourself first. Use AI only for hints, verification, or after-the-fact explanation.
- Cross-check every derivation and numerical result against your textbook, lecture notes, or a second independent source.
- Cite AI assistance when required by your institution (many universities now ask students to disclose generative-AI use).
- Treat AI output as a draft conversation partner, not an authority. If the explanation conflicts with course material, ask the instructor.
- Do not submit AI-generated solutions as your own work.

## 2. Database Management Systems (DBMS)

### Practical examples

- Schema design: “Given these business requirements for a university registration system, propose a normalized relational schema up to 3NF. Explain functional dependencies and why each table is in 3NF.”
- SQL practice: Paste a schema and ask for progressively harder queries (joins, subqueries, window functions, CTEs). Request both the query and an explanation of the execution plan.
- Debugging: Provide a failing query and the error message; ask the AI to diagnose and rewrite while explaining the root cause (e.g., Cartesian product, missing join condition, aggregate misuse).
- Conceptual mapping: “Translate this ER diagram description into relational tables, including primary and foreign keys. Then show the reverse: given tables, reconstruct possible ER relationships.”
- Advanced topics: Ask for explanations of indexing strategies, transaction isolation levels, or concurrency control with concrete lock-based scenarios and potential anomalies (dirty read, phantom read).

### Study strategies

- Design–implement–critique cycle: Write a schema or query yourself → ask AI to review it against normalization rules or performance considerations → revise → re-test.
- Progressive difficulty: Start with simple SELECT/WHERE, then force multi-table joins, then aggregation + HAVING, then recursive CTEs or window functions.
- Mental model building: Request side-by-side comparisons (e.g., "How does a B+ tree index differ from a hash index for range queries?") and follow up with "Show me a small example table and the resulting index structure conceptually."
- Error-driven learning: Deliberately introduce common mistakes (missing GROUP BY, incorrect outer joins) and ask the AI to explain the symptoms and fixes.
- Project support: Use AI to brainstorm alternative designs or to generate realistic sample data for testing, but implement and optimize the final schema and queries yourself.

### Benefits

- Rapid feedback on SQL correctness and style without needing a live database for every iteration.
- Clear explanations of abstract concepts (normalization, ACID properties, query optimization) tied to concrete examples.
- Ability to explore design trade-offs (normalization vs. denormalization for performance) quickly.
- Practice with modern SQL features that textbooks may cover lightly.

### Limitations

- AI-generated schemas or queries may violate subtle integrity constraints or perform poorly at scale; they are rarely production-ready.
- Dialect differences (PostgreSQL vs. MySQL vs. Oracle) can lead to non-portable or incorrect syntax.
- Execution-plan explanations are approximate; real DBMS optimizers depend on statistics and configuration you must verify yourself.
- Complex multi-statement transactions or concurrency scenarios are easy for AI to oversimplify.

### Responsible use

- Never paste proprietary or exam schemas/queries into public AI tools if your course or institution forbids it.
- Implement and test every suggested query or schema in your own DBMS environment (SQLite, PostgreSQL, MySQL, etc.).
- Document design decisions yourself; do not copy AI justifications verbatim into assignments or reports.
- When the AI suggests an optimization, measure it (EXPLAIN ANALYZE) rather than accepting the claim.
- Disclose AI assistance according to course policy, especially for design projects and take-home exams.

## 3. Software Engineering

### Practical examples

- Requirements and design: "Given this informal description of a library management system, produce a list of functional and non-functional requirements, then sketch a high-level component diagram and sequence diagram for the borrow-book use case (describe them in text or PlantUML)."
- Design patterns and architecture: "Explain the Observer pattern with a concrete example relevant to a notification system. Show before-and-after code sketches and discuss trade-offs."
- Code review and refactoring: Paste a function or class and ask for readability, SOLID-principle, and potential-bug feedback; request a refactored version with explanation of each change.
- Testing: "Generate unit-test cases (including edge cases) for this method using pytest / JUnit style. Explain why each test is valuable."
- Process and tools: Ask for comparisons of Agile vs. Waterfall in a student-project context, or for a sample Git branching strategy and commit-message conventions for a team of four.

### Study strategies

- Iterative design practice: Write requirements or a UML description yourself → critique with AI → revise → implement a thin vertical slice → review the code with AI.
- Pattern recognition drills: Provide a code smell or design problem and ask the AI to identify applicable patterns, then implement the solution yourself.

- Testing-first mindset: Have the AI generate test cases from a specification before you write the implementation; use the tests as a specification check.
- Code reading and critique: Regularly feed short open-source snippets or your own previous assignments into the AI and practice evaluating the feedback critically.
- Project retrospectives: After a milestone, ask the AI to help structure a lessons-learned discussion based on the issues you encountered (without pasting private team data).

### Benefits

- Fast feedback on design quality, code style, and testing completeness.
- Exposure to a wide range of patterns, architectural styles, and best practices beyond a single textbook.
- Safe environment to explore "what would a better design look like?" without the cost of large rewrites.
- Support for documentation (README structure, API comments, design rationales) that many students under-emphasize.

### Limitations

- AI code and designs can be outdated, insecure, or ignore project-specific constraints (language version, library policies, performance targets).
- Generated UML or architecture descriptions are textual approximations; visual tools and team consensus remain essential.
- Over-acceptance of AI suggestions can produce homogeneous, "average" solutions that lack originality or deep understanding of trade-offs.
- AI is weak at large-scale system reasoning (distributed consistency, long-term maintainability, team process dynamics).

### Responsible use

- Treat AI as a junior pair-programmer or reviewer, not as the author of record. You remain accountable for every line of code and every design decision.
- Never submit AI-generated code or diagrams without substantial personal modification, testing, and understanding.
- Check licenses and security implications of any suggested libraries or patterns.
- Respect collaboration rules: if pair-programming or team work is required, do not offload individual contributions to AI.

- Disclose AI use in project reports or oral defenses when asked; be prepared to explain and defend every choice the AI influenced.
- For graded individual work, follow the course's explicit generative-AI policy (some courses ban it for coding assignments; others allow it with citation).

### Cross-Cutting Recommendations

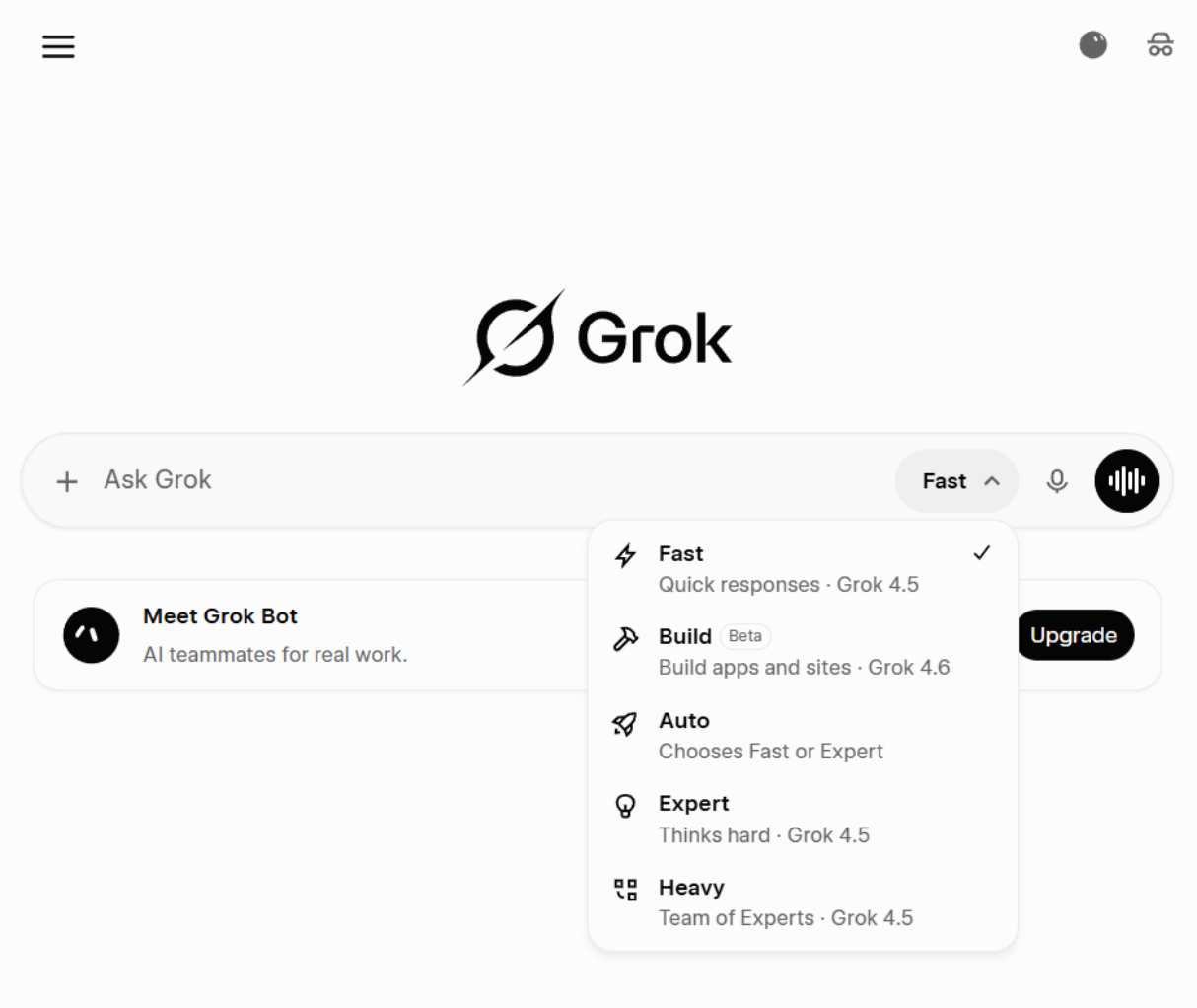
- **Active engagement first**: Always produce your own attempt (problem solution, schema, code, design) before consulting AI. The learning occurs in the struggle and the subsequent comparison.
- **Verification habit**: Cross-check AI output against primary sources (textbooks, official documentation, lecture notes, running code, statistical software). Use at least two independent methods when accuracy matters.
- **Prompt engineering for learning**: Prefer prompts that force explanation, comparison, or critique ("Explain why…", "Compare these two approaches…", "What are the weaknesses of this solution?") over pure generation.
- **Academic integrity**: Know your university's and each instructor's policy. When in doubt, ask. Disclose assistance when required. Plagiarism detectors and oral exams increasingly probe understanding.
- **Privacy and data**: Do not upload confidential course materials, unpublished exam questions, or personal data of classmates.
- **Complement, do not replace**: AI is excellent for practice volume, alternative explanations, and rapid feedback. It is a poor substitute for attending lectures, participating in discussions, solving problems under time pressure, and receiving human instructor feedback.
- **Metacognition**: Periodically ask yourself (and the AI) "What do I still not understand?" and "How would I explain this to a peer without notes?"

### Conclusion

Generative AI is a powerful accelerator for Probability & Statistics, DBMS, and Software Engineering when treated as an interactive tutor and critic rather than an answer machine. The highest returns come from using it to generate targeted practice, obtain alternative explanations, receive immediate feedback on your own work, and explore edge cases—while

rigorously verifying every output and retaining full responsibility for understanding and submitted work.

Used responsibly, it can deepen conceptual mastery, improve problem-solving speed, and free time for higher-order thinking. Used irresponsibly, it produces fragile knowledge and integrity risks. The difference lies in whether you remain the active learner directing the tool, or whether the tool becomes a substitute for thinking.



## Reflection

After trying the different Generative AI tools, I found that they can be very useful for university learning, especially for subjects such as Probability and Statistics, Database Management Systems, and Software Engineering. One useful thing I discovered is that the quality of the answer depends greatly on how the prompt is written. Instead of asking AI to simply give an answer, I can ask it to explain concepts step by step, identify my mistakes, generate practice questions, or quiz me without immediately showing the answer. This can make learning more interactive and help me understand difficult topics. The reports also highlighted that AI can be used for debugging SQL and programming code, explaining mathematical concepts with real-life examples, and preparing for exams. In my opinion, **Qwen Thinking produced the best overall results**. Its report was well structured, clear, and provided a wide range of practical examples for all three subjects. I particularly found its suggestions about using the Feynman Technique for understanding concepts, the Socratic Method for solving problems, and AI-generated quizzes for revision very useful. DeepSeek R1 also produced a detailed and useful report, especially with its practical examples and emphasis on using AI to identify mistakes rather than simply providing answers.

Overall, I learned that Generative AI is most effective when it is used as a **co-pilot or personal tutor rather than an autopilot**. AI responses should still be verified using lecture notes, textbooks, and other reliable sources because AI can produce incorrect information. Students should also avoid becoming overdependent on AI and should maintain academic integrity by doing their own thinking and work.